



Departamento de Informática e Matemática Aplicada – DIMAp

Comunicação TCP

Prof. Nélio Cacho

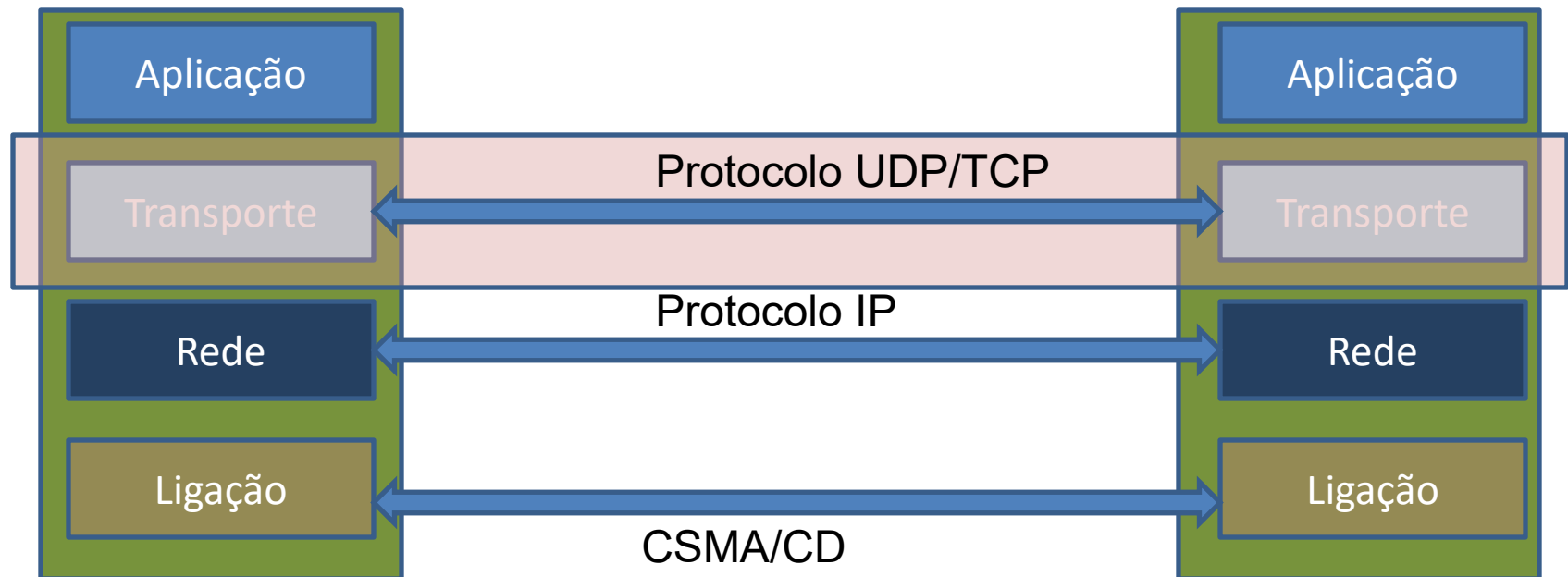
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

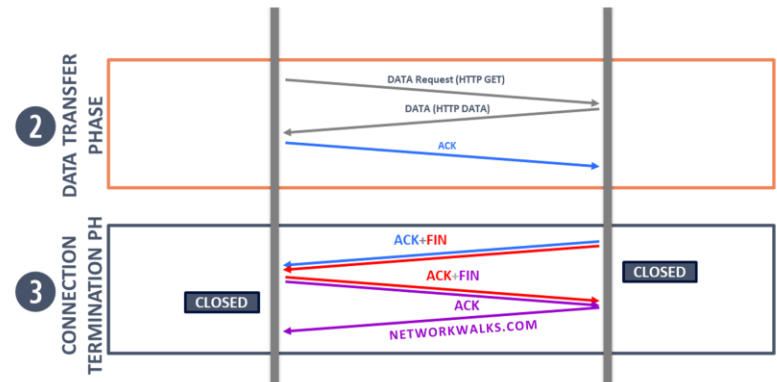
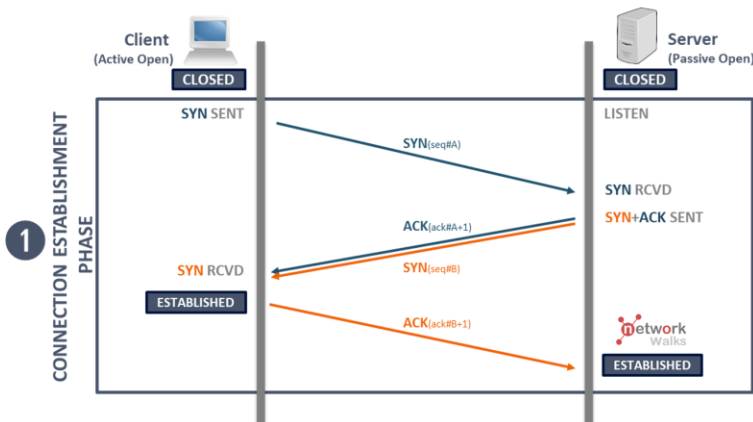
Como realizar a comunicação ?



The three way handshake

- O *three way handshake* é crucial para garantir a confiabilidade da conexão TCP. Este processo permite que ambos os dispositivos confirmem a recepção dos pacotes iniciais antes de começar a transmitir dados. Isso ajuda a evitar problemas como a perda de pacotes e a duplicação de dados, que podem ocorrer em conexões não confiáveis.

TCP 3-WAY HANDSHAKE PROCESS

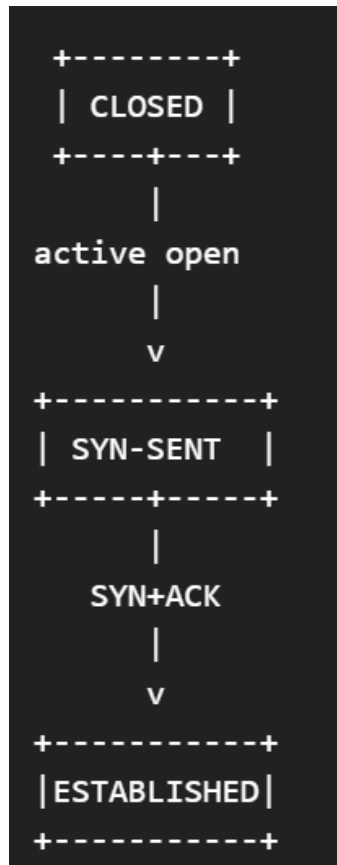


Vantagens do TCP

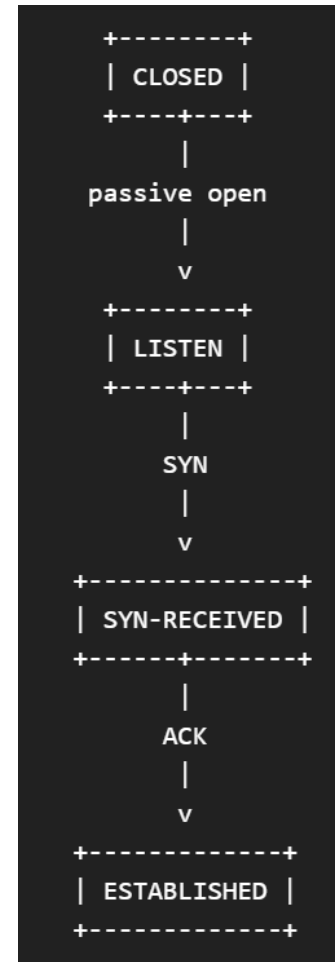
- Controle automático de erro
- Confiabilidade – Dados são entregues na ordem em que são enviados
- Facilidade de Uso: Não existe a necessidade de enviar pacotes de dados separadamente.

Ciclo de Vida da Conexão TCP

- No lado do Cliente:



No lado do Servidor



Como acompanhar o status da sua conexão

```
C:\Users\nelio>netstat -an | findstr /C:"Proto" /C:":8080 "
```

Proto	Endereço local	Endereço externo	Estado
TCP	0.0.0.0:8080	0.0.0.0:0	LISTENING
TCP	:::8080	:::0	LISTENING

```
PS C:\Windows\system32> Get-NetTCPConnection -LocalPort 8080
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	AppliedSetting	OwningProcess
::	8080	::	0	Listen		33816

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	0.0.0.0:36045	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:111	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:53363	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:25	0.0.0.0:*	LISTEN
tcp	0	0	128.133.190.219:34077	4.71.104.187:80	TIME_WAIT
tcp	0	0	128.133.190.219:43346	79.62.132.8:22	ESTABLISHED
tcp	0	0	128.133.190.219:875	128.133.190.43:2049	ESTABLISHED
tcp6	0	0	:::22	:::*	LISTEN

Interação TCP Servidor

Servidor começa por criar um TCP Server Socket...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
ServerSocket servSock = new ServerSocket(servPort, backlog);
```

Cliente

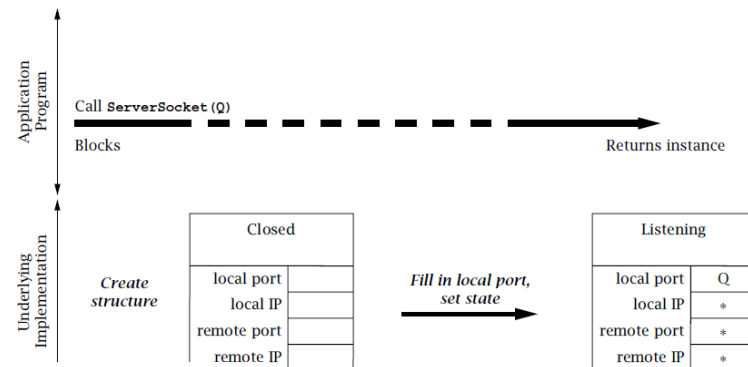
1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Conectando pelo lado do Servidor

- Primeiro, o servidor cria uma instância de `ServerSocket` associada à sua **porta conhecida** (*well-known port*) — neste caso, **Q**.
- A implementação do socket cria uma estrutura interna de socket correspondente à nova instância de `ServerSocket` e preenche **Q** como a porta local e o **endereço curinga** (*wildcard address*) especial como o endereço IP local.
- O estado do socket é definido como **LISTENING**, indicando que ele está pronto para aceitar solicitações de conexão de entrada destinadas à sua porta.
- O servidor pode agora chamar o método **`accept()`** do `ServerSocket`, que **bloqueia a execução** até que o *handshake* de abertura do TCP seja concluído com algum cliente e uma nova conexão seja estabelecida.



```
Active Internet connections
Proto Recv-Q Send-Q Local Address
```

```
Foreign Address
```

```
State
```

```
tcp          0      0 0.0.0.0:Q
```

```
0.0.0.0:0
```

```
LISTENING
```

Interação TCP Servidor

```
for (;;) {  
    Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

Servidor fica bloqueado esperando conexões

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

Um cliente então resolve estabelecer uma comunicação com o servidor...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

```
Socket socket = new Socket(server, servPort);
```

Cliente

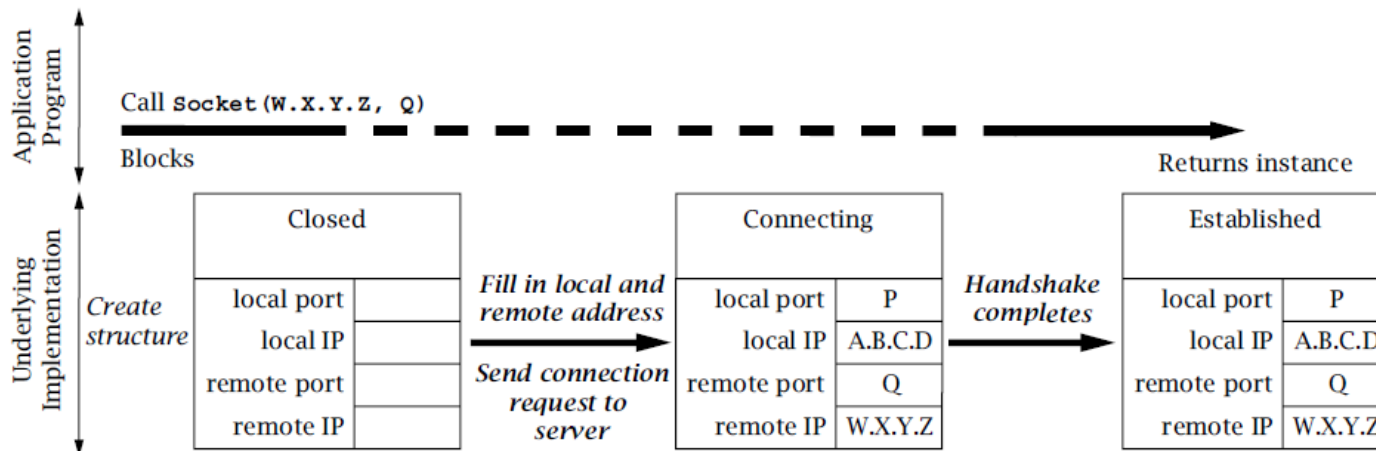
1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Conectando pelo lado do Cliente

- Quando se chama:
 - `Socket socket = new Socket(W.X.Y.Z, Q);`
- `new Socket()` não é uma operação instantânea. É uma chamada de método que desencadeia um protocolo distribuído entre dois computadores.



Conectando pelo lado do Cliente

- Depois que a mensagem inicial do handshake é enviada e antes que a resposta do servidor seja recebida, a saída do comando **netstat** no host do cliente seria semelhante a:

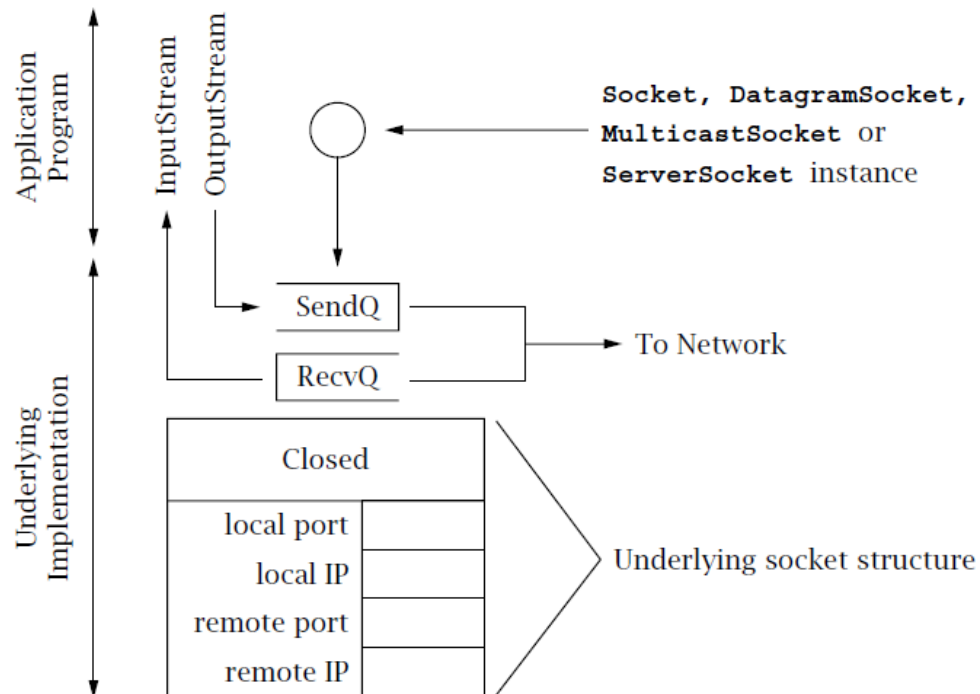
```
Active Internet connections
Proto Recv-Q Send-Q Local Address   Foreign Address  State
tcp      0      0 A.B.C.D:P      W.X.Y.Z:Q       SYN_SENT
```

- Se o servidor não estiver aceitando conexões — por exemplo, se não houver nenhum programa associado à porta especificada no destino — o TCP do lado do servidor enviará uma **mensagem de rejeição** em vez de uma confirmação, e o construtor lançará uma `IOException` quase imediatamente.
- Caso contrário, depois que o cliente receber uma **resposta positiva do servidor**, a saída do comando **netstat** será semelhante a:

```
Active Internet connections
Proto Recv-Q Send-Q Local Address   Foreign Address  State
tcp      0      0 A.B.C.D:P      W.X.Y.Z:Q       ESTABLISHED
```

Interação TCP Cliente

```
Socket socket = new Socket(server, servPort);
```



Interação TCP Cliente

```
OutputStream out = socket.getOutputStream();  
out.write(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

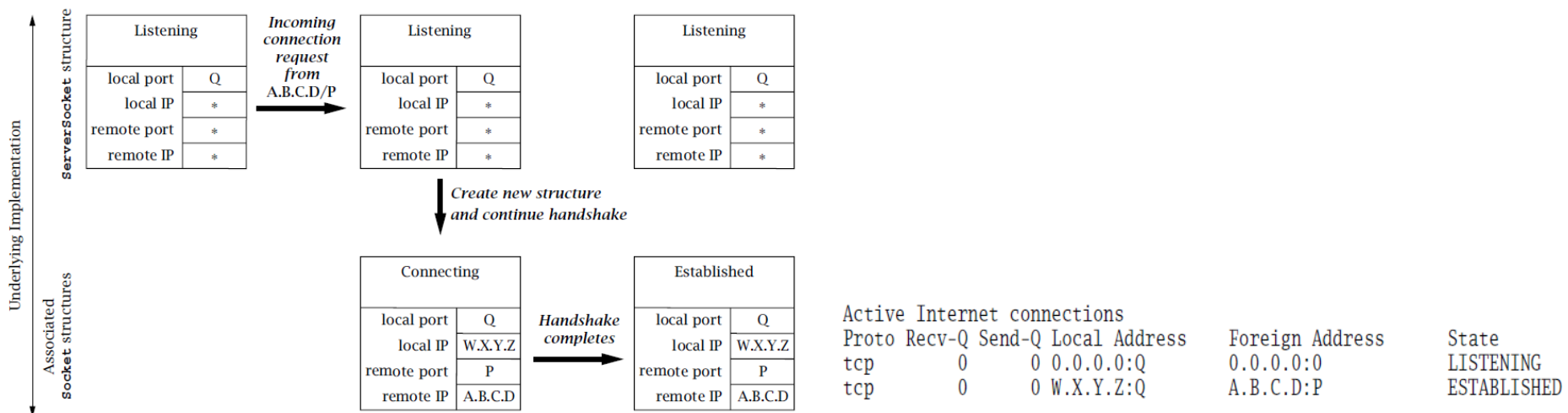
Conectando pelo lado do Servidor

- Quando a solicitação de conexão chega do cliente, uma **nova estrutura de socket** é criada para essa conexão. Os endereços do novo socket são preenchidos com base no pacote recebido:
 - o **endereço IP e a porta de destino** do pacote (W.X.Y.Z e Q, respectivamente) tornam-se o **endereço IP local e a porta local**;
 - o **endereço IP e a porta de origem** do pacote (A.B.C.D e P) tornam-se o **endereço IP remoto e a porta remota**.
- Observe que o **número da porta local do novo socket é sempre o mesmo da porta do ServerSocket**.
- O estado do novo socket é definido para indicar que ele está **“Conectando”** — tecnicamente chamado de **SYN_RCVD** no lado do servidor — e ele é adicionado a uma lista de sockets **ainda não completamente conectados**, associada à estrutura de socket do ServerSocket.
- Observe que o próprio ServerSocket **não muda de estado**, nem qualquer uma das suas informações de endereço é alterada.
- Nesse momento, a saída do comando netstat deve mostrar **tanto o socket original, que está aguardando conexões, quanto o novo socket criado para a conexão**:

```
Active Internet connections
Proto Recv-Q Send-Q Local Address      Foreign Address    State
tcp        0      0 0.0.0.0:Q          0.0.0.0:0         LISTENING
tcp        0      0 W.X.Y.Z:Q         A.B.C.D:P         SYN_RCVD
```

Conectando pelo lado do Servidor

- Quando o cliente envia os dados, é criada uma nova estrutura interna de socket do lado do servidor que envia de volta ao cliente uma mensagem de confirmação do handshake TCP.
- No entanto, o TCP do servidor não considera o handshake concluído até que a terceira mensagem do three-way handshake seja recebida do cliente.
- Quando essa mensagem finalmente chega, o estado da nova estrutura é alterado para ESTABLISHED e, somente então, ela é movida para uma lista de estruturas de sockets associada à estrutura do ServerSocket.
- Essa lista representa as conexões estabelecidas que estão prontas para serem aceitas (accept()) por meio do ServerSocket.
- Se a terceira mensagem do handshake não chegar, a estrutura que estava no estado “Connecting” será eventualmente excluída.



Interação TCP Servidor

```
InputStream in = clntSock.getInputStream();  
recvMsgSize = in.read(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente/Servidor

`close(sock);`

`close(clntSocket)`

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Fechando Conexão

- O TCP permite que uma aplicação encerre uma conexão **sem perder dados que ainda estejam em trânsito**.
- Além disso, o TCP permite que cada direção da comunicação seja encerrada **independentemente**.
- Quando o método `close()` é chamado, o TCP primeiro garante que os dados pendentes sejam enviados.
- O FIN **não é entregue diretamente à aplicação remota**.
- Em vez disso, quando a aplicação remota tentar ler: `InputStream.read()` e todos os dados anteriores já tiverem sido consumidos, o `read()` retornar -1.

Status do TCP

Estado	Significado	
CLOSED	Não existe conexão TCP ativa	
LISTEN	Servidor está aguardando solicitações de conexão	
SYN-SENT	O endpoint enviou SYN e está esperando resposta	
SYN-RECEIVED	Recebeu SYN , enviou SYN+ACK e aguarda confirmação	
ESTABLISHED	Conexão estabelecida; dados podem ser transmitidos	
FIN-WAIT-1	Enviou FIN e aguarda ACK ou outro FIN	
FIN-WAIT-2	Seu FIN foi confirmado; aguarda o FIN do outro lado	
CLOSE-WAIT	Recebeu FIN do outro lado; aplicação ainda não fechou	
CLOSING	Ambos enviaram FIN e aguardam confirmação	
LAST-ACK	Enviou seu FIN após receber FIN ; aguarda ACK	
TIME-WAIT	Conexão terminou, mas TCP aguarda antes de eliminar completamente o estado	

Enviando Instancias de Classes

```
public class MsgTCP implements Serializable{  
    private int proc;  
    private int mem;  
    private int nclients;  
    private String hostname;  
}
```

Enviando Instancias de Classes

```
Socket conexao = new Socket ("localhost", SERVICE_PORT);
```

```
ObjectOutputStream output = new  
    ObjectOutputStream(conexao.getOutputStream());
```

```
MsgTCP msg = new MsgTCP();
```

```
msg.setMem(20);
```

```
output.writeObject(msg);
```

```
output.flush();
```

Exemplo Cliente

```
public class TCPClient {  
  
    Run | Debug | Run main | Debug main  
    public static void main(String args[]) {  
        Socket conexao = null;  
        ObjectOutputStream output = null;  
        ObjectInputStream input = null;  
        try {  
            System.out.println(x: "TCP Client Bank Started");  
            conexao = new Socket(host: "localhost", port: 8080);  
            output = new ObjectOutputStream(conexao.getOutputStream());  
            output.writeObject(obj: "criar;1;1000");  
            output.flush();  
            input = new ObjectInputStream(conexao.getInputStream());  
            String msg = (String) input.readObject();  
            System.out.println("Retorno do Servidor:"+msg);  
  
        } catch (UnknownHostException e) {  
            e.printStackTrace();  
        } catch (IOException e) {  
            e.printStackTrace();  
        } catch (ClassNotFoundException e) {  
            e.printStackTrace();  
        } finally {  
            try {  
                input.close();  
                output.close();  
                conexao.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
        System.out.println(x: "TCP Client Bank Finished");  
    }  
}
```

Exemplo Servidor – Mono Thread com Objeto

```
public static void main(String args[]) {  
    System.out.println(x: "TCP Server Bank Started");  
    ServerSocket server = null;  
    Banco banco = new Banco();  
    ProcessPayload processpayload = new ProcessPayload(banco);  
    try {  
        server = new ServerSocket(port: 8080);  
    } catch (IOException e2) {  
        e2.printStackTrace();  
    }  
    while (true) {  
        try {  
            Socket conexao = server.accept();  
            ObjectInputStream input = new ObjectInputStream(conexao.getInputStream());  
            String msg = (String) input.readObject();  
            System.out.println("Operação recebida:"+msg);  
  
            String reply = processpayload.processData(msg,conexao.getInetAddress().toString());  
  
            ObjectOutputStream output = new ObjectOutputStream(conexao.getOutputStream());  
            output.writeObject(reply);  
            output.flush();  
            output.close();  
            conexao.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        } catch (ClassNotFoundException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
    }  
}
```

Exemplo Servidor – Tratando Protocolo de Aplicação

```
public class ProcessPayload {
    private Banco banco;
    public ProcessPayload(Banco bank) {
        this.banco = bank;
    }
    public String processData(String msg, String Ip) {

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        String operacao = null;
        int conta = 0;
        int valor = 0;

        StringTokenizer tokenizer = new StringTokenizer(msg, ";");
        while (tokenizer.hasMoreElements()) {
            operacao = tokenizer.nextToken();
            conta = Integer.parseInt(tokenizer.nextToken());
            valor = Integer.parseInt(tokenizer.nextToken().trim());
        }
        String reply = "Operacao realizada:" + operacao + "-" + Ip;
        String resultadoOp = reply;
        switch (operacao) {
            case "criar":
                banco.addConta(conta);
                break;
            case "depositar":
                banco.depositar(conta, valor);
                break;
            case "saldo":
                resultadoOp = "Saldo é R$" + banco.saldo(conta);
                break;
        }

        System.out.println(resultadoOp);
        return resultadoOp;
    }
}
```

Exemplo Servidor – Mono Thread com Rawdata

```
public class TCPServerRaw {  
    Run | Debug | Run main | Debug main  
    public static void main(String args[]) {  
        System.out.println(x: "TCP Server Bank Started");  
        ServerSocket server = null;  
        Banco banco = new Banco();  
        ProcessPayload processpayload = new ProcessPayload(banco);  
        try {  
            server = new ServerSocket(port: 8080, backlog: 300);  
        } catch (IOException e2) {  
            e2.printStackTrace();  
        }  
        while (true) {  
            try {  
                Socket conexao = server.accept();  
                BufferedReader input = new BufferedReader(new InputStreamReader(conexao.getInputStream()));  
                String msg = input.readLine();  
                System.out.println("Operação recebida:"+msg);  
  
                String reply = processpayload.processData(msg, conexao.getInetAddress().toString());  
            }  
        }  
    }  
}
```

Exemplo Servidor – Multi-Thread com Rawdata

```
public class TCPServerRawNThreadPlatform {  
    private static final int THREAD_POOL_SIZE = 300;  
    private static final ExecutorService threadPool = Executors.newFixedThreadPool(THREAD_POOL_SIZE);  
  
    Run | Debug | Run main | Debug main  
    public static void main(String args[]) {  
        System.out.println(x: "TCP Server Bank Started");  
        Banco banco = new Banco();  
        ProcessPayload processPayload = new ProcessPayload(banco);  
  
        try {  
            ServerSocket server = new ServerSocket(port: 8080, backlog: 1000);  
            while (true) {  
                try {  
                    Socket conexao = server.accept();  
                    threadPool.execute(() -> handleClient(conexao, processPayload));  
                } catch (IOException e) {  
                    e.printStackTrace();  
                }  
            }  
        }  
    }  
}
```

Como modificar o comportamento de sockets TCP em Java?

Por que existem?

Socket options modificam o comportamento do socket e podem afetar desempenho, latência, consumo de recursos e robustez.

Regra geral

Não altere opções sem uma razão clara. Uma configuração que ajuda um cenário pode prejudicar outro.

Exceção importante

SO_TIMEOUT: aplicações TCP devem tratar timeouts para não ficarem bloqueadas indefinidamente.

Detectar conexões TCP que ficaram silenciosamente inativas

- Normalmente, uma conexão TCP pode permanecer aberta mesmo sem transmitir dados:
- Se o cliente sofrer uma falha abrupta, o servidor pode continuar mantendo recursos associados à conexão.
- **SO_KEEPALIVE:** Permite que o TCP envie **sondagens** (*keepalive probes*) para verificar se o outro endpoint ainda está ativo.
- `socket.setSoKeepAlive(true);`
- **Atenção**
 - O intervalo das sondagens normalmente é controlado pelo sistema operacional.
 - A aplicação Java não controla diretamente essa frequência através de `setSoKeepAlive()`.
 - Muitos sistemas preferem implementar **timeouts no nível da aplicação**.

O que acontece quando `close()` é chamado?

- Pode haver dados ainda aguardando transmissão quando o socket é fechado.
- **SO_LINGER** controla **quanto tempo o sistema deve aguardar durante o fechamento para tentar transmitir dados pendentes**.
- Exemplo: `socket.setSoLinger(true, 50);`
- Neste caso: habilita *linger* com duração de 50 segundos.
- **Cuidado**
 - Alterar **SO_LINGER** pode modificar significativamente o comportamento de `close()`.

TCP_NODELAY

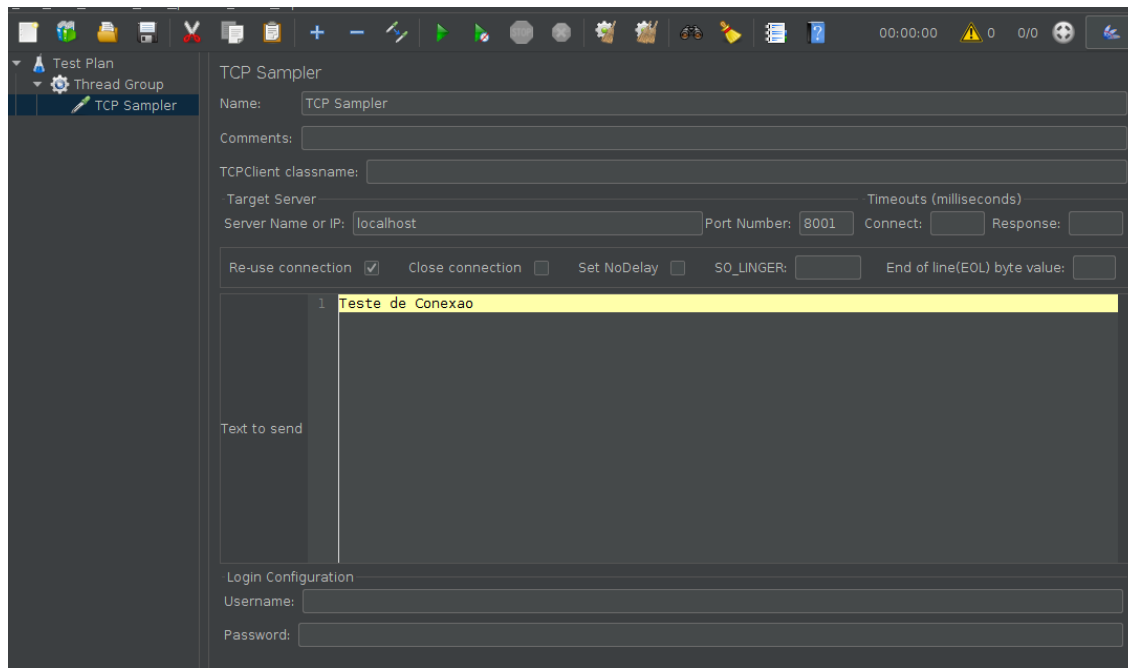
- **TCP_NODELAY** controla se o algoritmo de Nagle está habilitado.
- O Algoritmo de Nagle reduz o overhead da rede agrupando pequenas quantidades de dados em segmentos TCP maiores antes de enviá-los, em troca de potencial aumento de latência.
- Em Java:
 - `socket.setTcpNoDelay(true);`
- **Quando desabilitar?**
 - Pode ser interessante para aplicações sensíveis à latência, como:
 - aplicações interativas;
 - comunicação com mensagens pequenas;
 - alguns protocolos de baixa latência.
- **Mas cuidado!**
 - Desabilitar Nagle pode aumentar:
 - número de pacotes;
 - overhead;
 - consumo de banda.

A opção mais importante para aplicações distribuídas - SO_TIMEOUT

- Por padrão, uma leitura do InputStream pode bloquear enquanto espera dados.
- Se o peer falhar ou parar de enviar dados, a aplicação pode ficar travada.
- SO_TIMEOUT define quanto tempo uma operação de leitura pode esperar.
- Quando expira, Java lança SocketTimeoutException, uma subclasse de InterruptedIOException.
- A aplicação decide o que fazer: retry, informar o usuário, fechar a conexão etc.
- `socket.setSoTimeout(5_000); // 5 segundos`

Testar Serviço TCP com JMeter

- Para criar um Sampler TCP, clicar com botão direito no ThreadGroup, "add"-> "Sampler"-> "TCP Sampler".



Criar Sampler TCP

Test Plan
Thread Group
TCP Sampler

TCP Sampler

Name: TCP Sampler

Comments:

TCPClient classname:

Target Server

Server Name or IP: localhost Port Number: 8001

Re-use connection ☒ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

1 Teste de Conexao

2

Text to send

Login Configuration

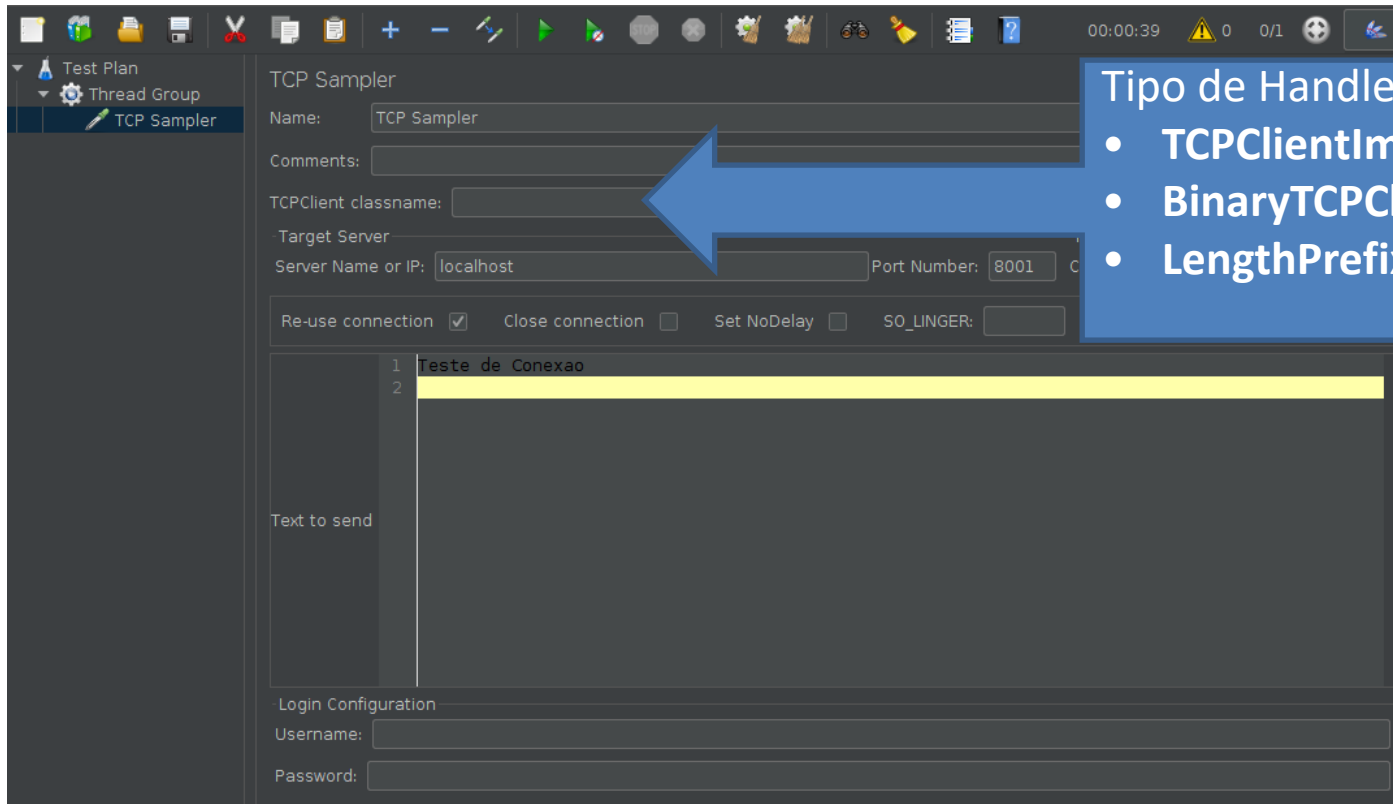
Username:

Password:

Endereço e Porta que o servidor TCP foi iniciado

Colocar dado a ser enviado

Criar Sampler TCP



Tipo de Handler:

- TCPClientImpl (padrão)
- BinaryTCPClientImpl
- LengthPrefixedBinaryTCPClientImpl

Criar Sampler TCP

- Um novo Handler "must extend":
 - **org.apache.jmeter.protocol.tcp.sampler.TCPClient**

TCPClientImpl

This implementation is fairly basic. When reading the response, it reads until the end of line byte, if this is defined by setting the property **tcp.eolByte**, otherwise until the end of the input stream. You can control charset encoding by setting **tcp.charset**, which will default to Platform default encoding.

BinaryTCPClientImpl

This implementation converts the GUI input, which must be a hex-encoded string, into binary, and performs the reverse when reading the response. When reading the response, it reads until the end of message byte, if this is defined by setting the property **tcp.BinaryTCPClient.eomByte**, otherwise until the end of the input stream.

LengthPrefixedBinaryTCPClientImpl

This implementation extends **BinaryTCPClientImpl** by prefixing the binary message data with a binary length byte. The length prefix defaults to 2 bytes. This can be changed by setting the property **tcp.binarylength.prefix.length**.

Atividade

- Dentro do JMeter, mude os parâmetros de teste e registre os resultados.

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

TCP Sampler

Name:

Comments:

TCPClient classname:

Target Server

Server Name or IP: Port Number:

Timeouts (milliseconds)

Connect: Response:

Re-use connection ☐ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value: